Full-Stack Developer: Take-Home Assignment

4-5 Years Experience | Maximum Timebox 9 Hours

Objective

This assignment assesses your ability to independently design and deliver a production-minded full-stack feature using React, Python, Django, Django REST Framework, PostgreSQL, Redis and Celery. We will evaluate product judgment, technical depth, data and API design, reliability, testing and the clarity of your engineering decisions.

Assignment Brief

Implement the screens and primary interactions shown in the wireframe. Treat it as the functional and structural reference rather than a pixel-perfect visual specification. Where behaviour or business rules are not explicitly shown, make reasonable assumptions and document them in your README.

Core Functional Requirements

- Implement the complete flow represented in the wireframe, including the visible forms, actions and **navigation between screens**.
- Provide an API-backed record **listing** with server-side **pagination**, **search**, **filtering** and **sorting**, using enough seed data to demonstrate each behaviour.
- Support create, view and update operations required by the flow, with clear client-side and server-side validation.
- Enforce valid business-state transitions and reject invalid transitions with a clear API response.
- Handle loading, empty, success, **validation**, **permission** and unexpected API-error states.
- **Support desktop and mobile widths** and follow semantic HTML and basic accessibility practices.

Technical Requirements

Backend and Data

- Use **Python**, **Django** and **Django REST Framework** with **PostgreSQL**. Include migrations and a reproducible way to load sample data.
- Design a relational model with meaningful relationships, database constraints and at least one justified index.
- Use appropriate REST semantics, HTTP status codes, serializer validation and a consistent error-response structure.
- Implement authentication and at least one meaningful authorization rule that restricts an action or data based on the authenticated user or role.
- Use **Redis** and **Celery** for one meaningful asynchronous operation, such as a simulated notification or audit-processing task. Include retry or failure handling and document how duplicate execution is handled.

Engineering Depth

Implement any two of the following and explain your choices in the README:

- Preserve a **history or audit record** of important changes, including timestamp and action performed.
- Use a database transaction where data consistency requires multiple related writes.
- Demonstrate an intentional query optimization, such as select_related, prefetch_related or aggregation.
- Add useful logging around a critical API action and the asynchronous task without logging sensitive data.

Frontend

- Use **React** and **Material UI** for core UI elements, with **reusable components** and a clear feature-level structure.
- Integrate with your own APIs and keep pagination, filters, sorting and search behaviour consistent with the backend.
- Provide clear form feedback and prevent accidental duplicate submissions while a request is in progress.
- Keep application state predictable and avoid unnecessary data fetching or re-rendering.

Environment and Delivery

- Provide **Docker Compose configuration** for the web application, PostgreSQL, Redis and a Celery worker. The project should start with clearly documented commands.
- Use environment variables for configuration and provide a safe example environment file without credentials or secrets.
- Include a concise **AWS deployment note** describing the services you would use, configuration and secret management, static or media handling, Celery worker deployment, scaling, monitoring and failure recovery. A live AWS deployment is not required.

Testing Expectations

Include at least four meaningful automated tests covering:

- A successful API flow or a business rule or invalid state transition
- Authentication or authorization behaviour
- The Celery task, including failure or retry behaviour
- An important frontend component or user interaction

A large test suite is not expected; choose tests that protect the most important behaviour.

What We Will Evaluate

- Interpretation of the wireframe and product requirements
- Django, DRF, API and relational data implementation
- React structure, Material UI usage, reusable components, paginated API-backed ticket listing and frontend-backend integration
- Validation, error handling, responsiveness and relevant application states
- Code quality, testing, documentation and ability to explain decisions

Deliverables

1. **Source code.** Submit an accessible GitHub repository or a ZIP file. Include migrations, Docker configuration and sample-data setup.
2. **README.** Include prerequisites, setup and run instructions, test commands, API endpoints, data-model explanation, architecture decisions, assumptions, limitations, actual time spent, query optimization, Celery and Redis design, and what you would improve with more time.
3. **AWS deployment note.** Include the production deployment approach requested above; this may be part of the README.
4. **Optional demonstration.** You may include screenshots or a short screen recording of the implemented flow.

Do not commit dependency folders, credentials, secrets or environment-specific files.

AI-Assisted Development

You may use AI-assisted development tools. Disclose the tools used and how they supported your work in the README. You should be prepared to explain, debug and modify all submitted code during the follow-up technical discussion.

Submission

Please send your deliverables to **sneha.rachel@urbanvault.in** by the deadline communicated in the assignment email. The assignment itself should be **timeboxed** to a **maximum** of **9 hours**.